AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Założenia programu 3

1.2. Narzędzia i wymagania dla systemu Git 3

2. Analiza problemu **4**

2.1. Jak działa lista dwukierunkowa na stercie 4

2.2. Wykorzystane wzorce projektowe 4

2.3. Podsumowanie operacji na liście 5

3. Projektowanie **6**

3.1. Wykorzystane narzędzia i biblioteki 6

3.2. Ustawienia kompilatora i budowanie projektu 6

3.3. Diagram klas i schemat działania algorytmu 7

3.4. Sposób używania narzędzi Git, GitHub oraz Doxygen 7

4. Implementacja **9**

4.1. Implementacja węzła i wzorca Factory 9

4.2. Implementacja klasy listy dwukierunkowej 10

4.3. Implementacja wzorca Iteratora i testów w funkcji main 11

4.4. Przebieg testów z systemem Git 13

5. Wnioski **13**

Literatura **15**

Spis rysunków **15**

Spis tabel **16**

Spis listingów **17**

1. Ogólne określenie wymagań

Celem projektu jest napisanie w języku C++ programu obsługującego listę dwukierunkową, która działa na stercie. Oprócz samego kodu ważną częścią zadania jest praca z systemem kontroli wersji Git (wraz z serwisem GitHub) oraz wygenerowanie dokumentacji w programie Doxygen.

1.1. Założenia programu

W projekcie należało spełnić następujące wymagania dotyczące kodu:

- **Podział na pliki:** każda klasa musi znajdować się w osobnym pliku nagłówkowym (`.h`) oraz źródłowym (`.cpp`).
- **Pamięć dynamiczna:** wszystkie elementy listy są tworzone na stercie za pomocą operatora `new` i usuwane operatorem `delete`.
- **Wzorce projektowe:** podczas pisania programu należało wykorzystać wzorzec fabryki (*Factory*) oraz iteratora (*Iterator*).

Lista dwukierunkowa musi udostępniać następujące metody:

- dodawanie elementu na początek, na koniec oraz pod wskazany indeks,
- usuwanie elementu z początku, z końca oraz spod wskazanego indeksu,
- wyświetlanie całej listy od początku oraz w odwrotnej kolejności,
- wyświetlanie następnego i poprzedniego elementu,
- czyszczenie całej listy.

Działanie wszystkich powyższych metod trzeba przetestować w funkcji `main`.

1.2. Narzędzia i wymagania dla systemu Git

Podczas pisania programu kod był na bieżąco zapisywany w repozytorium na platformie GitHub. W ramach projektu należało przetestować podstawowe scenariusze pracy z Gitem:

1. wykonanie co najmniej 5 commitów podczas pisania kodu,
2. cofnięcie się w historii zmian o dwa commity,

3. usunięcie jednego commita,
4. pobranie projektu z GitHuba do innego folderu, dopisanie nowej metody i wysłanie zmian na serwer, a następnie pobranie ich w pierwszym folderze i dalsza praca nad kodem,
5. celowe usunięcie pliku z projektu (tak, aby program przestał się kompilować) i przywrócenie ostatniej działającej wersji z GitHuba.

2. Analiza problemu

Zwykła tablica w C++ ma stały rozmiar i ciężko dodawać do niej nowe elementy w środku lub na początku. Lista dwukierunkowa rozwiązuje ten problem, ponieważ składa się z osobnych węzłów łączonych za pomocą wskaźników, a jej rozmiar może się dynamicznie zmieniać w trakcie działania programu.

2.1. Jak działa lista dwukierunkowa na stercie

Każdy element (węzeł) listy przechowuje trzy rzeczy:

- wartość (w naszym przypadku liczbę całkowitą `int`),
- wskaźnik `next` pokazujący na następny element listy,
- wskaźnik `prev` pokazujący na poprzedni element listy.

Cała lista pamięta wskaźnik na swój pierwszy element (`head`) oraz ostatni (`tail`). Dzięki temu można łatwo przeglądać ją w obie strony – od początku do końca oraz od końca do początku.

Ponieważ lista działa na stercie, każdy nowy węzeł jest tworzony dynamicznie. Bardzo ważne jest tutaj pilnowanie pamięci – przy usuwaniu elementu lub czyszczeniu całej listy trzeba zwolnić pamięć operatorem `delete`, żeby nie dopuścić do wycieków pamięci.

2.2. Wykorzystane wzorce projektowe

Zgodnie z instrukcją w programie zastosowano dwa wzorce projektowe:

- **Factory (Fabryka):** zamiast wpisywać `new` bezpośrednio w metodach listy, tworzeniem nowych węzłów i samej listy zajmuje się osobna klasa `ListFactory`. Dzięki temu kod jest bardziej uporządkowany.

- **Iterator:** osobna klasa, która przechowuje wskaźnik na aktualnie wybrany element listy. Pozwala ona na wygodne przechodzenie po liście krok po kroku oraz wyświetlanie następnego i poprzedniego elementu bez ręcznego operowania na wskaźnikach w funkcji `main`.

2.3. Podsumowanie operacji na liście

W tabeli 2.1 krótko zestawiono najważniejsze operacje wykonywane na liście dwukierunkowej oraz sposób ich działania.

Tab. 2.1. Podstawowe operacje na liście dwukierunkowej

Operacja	**Opis działania w programie**
Dodawanie na początek / koniec	Tworzy nowy węzeł i przepina wskaźniki `head` lub `tail`. Działa bardzo szybko, bez przeglądania całej listy.
Dodawanie / usuwanie pod indeksem	Wymaga przejścia pętlą od początku listy do wybranego miejsca i odpowiedniego przepięcia wskaźników sąsiadów.
Wyświetlanie (przód / tył)	Przechodzi w pętli od `head` (przez `next`) lub od `tail` (przez `prev`) i wypisuje wartości na ekran.
Czyszczenie listy	Usuwa po kolei wszystkie węzły z pamięci za pomocą `delete`, aż lista będzie pusta.

3. Projektowanie

3.1. Wykorzystane narzędzia i biblioteki

Do zrealizowania projektu wykorzystano następujące narzędzia oraz technologie:

- **Język programowania:** C++ (standard C++11/C++17), który pozwala na programowanie obiektowe oraz ręczne zarządzanie pamięcią dynamiczną na stercie za pomocą operatorów `new` i `delete`.
- **Środowisko programistyczne (IDE):** Visual Studio Code działające w środowisku Linux, wyposażone we wtyczkę *C/C++ Extension Pack* oraz wbudowaną obsługę systemu Git.
- **Kompilator i debugger:** kompilator `g++` (z pakietu GCC) oraz debugger `gdb` pozwalający na śledzenie wykonywania programu krok po kroku.
- **Biblioteki dodatkowe:** w projekcie wykorzystano standardową bibliotekę wejścia/wyjścia `<iostream>`, która odpowiada za wypisywanie zawartości listy oraz komunikatów testowych w konsoli za pomocą strumienia `cout`. Cała struktura listy została napisana od podstaw, bez użycia gotowych kontenerów z biblioteki STL.
- **System kontroli wersji i dokumentacja:** system Git połączony ze zdalnym repozytorium w serwisie GitHub oraz program Doxygen (wraz z pakietem LaTeX) do automatycznego wygenerowania dokumentacji z komentarzy w kodzie.

3.2. Ustawienia kompilatora i budowanie projektu

Ze względu na wymóg podziału programu na osobne pliki dla każdej klasy, projekt składa się z wielu plików źródłowych (`.cpp`) oraz nagłówkowych (`.h`). Standardowa kompilacja pojedynczego pliku `main.cpp` kończy się błędem linkera, dlatego skonfigurowano kompilator `g++` w pliku `tasks.json` tak, aby kompilował i łączył wszystkie pliki jednocześnie.

Podczas kompilacji wykorzystano następujące polecenie i flagi:

```
g++ -g *.cpp -o program
```

Gdzie poszczególne opcje oznaczają:

- `-g` – dołącza informacje debugowania wymagane przez program `gdb`,
- `*.cpp` – wskazuje kompilatorowi wszystkie pliki źródłowe w katalogu projektu (`main.cpp`, `Node.cpp`, `DoublyLinkedList.cpp`, `ListFactory.cpp`, `ListIterator.cpp`),
- `-o program` – określa nazwę wynikowego pliku wykonywalnego.

W plikach nagłówkowych zastosowano dyrektywy preprocesora (`#ifndef`, `#define`, `#endif`), które zabezpieczają przed wielokrotnym dołączaniem tych samych klas podczas kompilacji.

3.3. Diagram klas i schemat działania algorytmu

Program został podzielony na cztery główne klasy współpracujące ze sobą. Na rysunku 3.1 przedstawiono schemat relacji między klasami (wzorzec Factory tworzy obiekty listy i węzłów, klasa listy zarządza węzłami, a iterator pozwala na poruszanie się po nich).

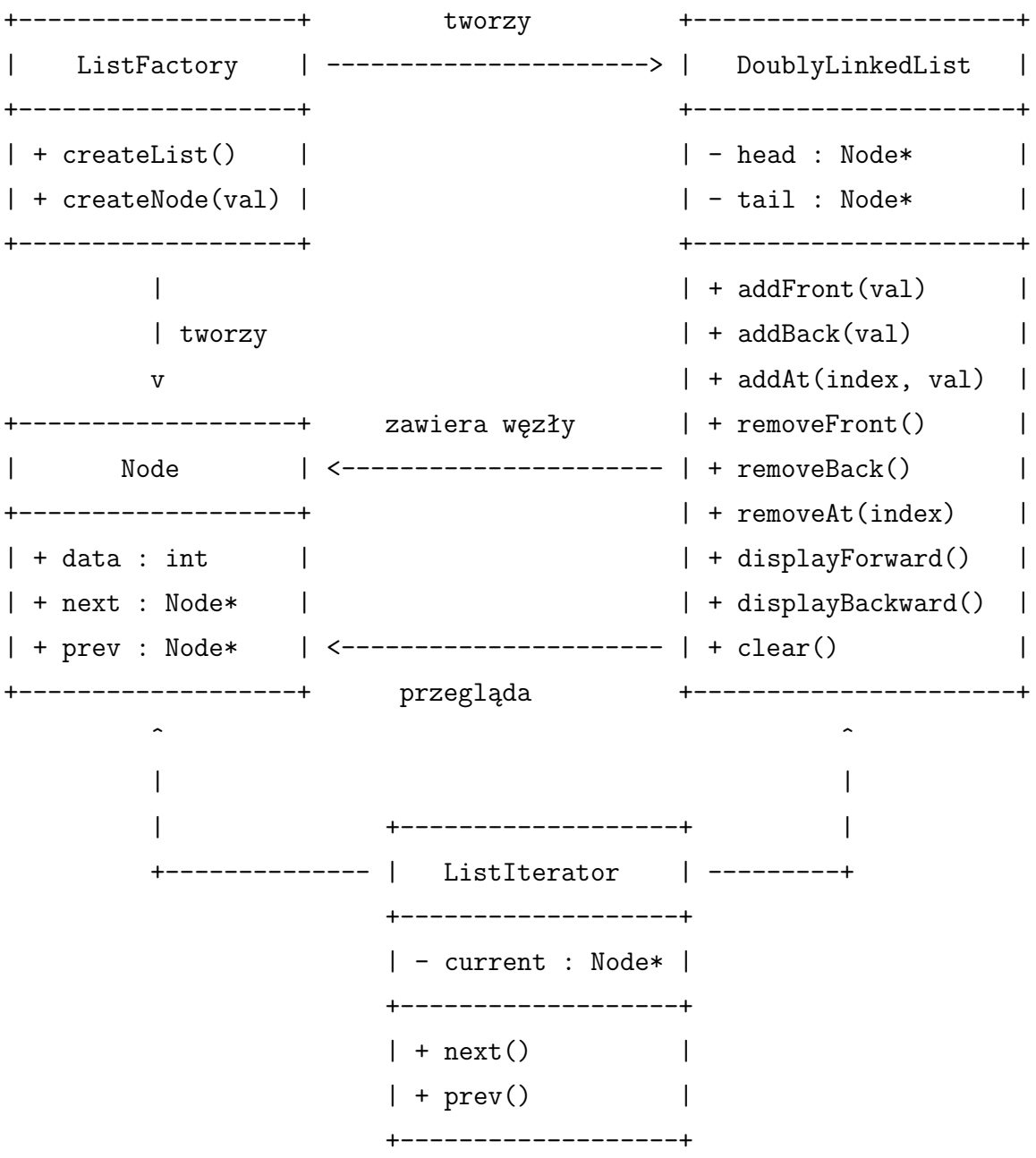
Działanie algorytmu listy dwukierunkowej opiera się na odpowiednim przepinaniu wskaźników `next` i `prev` pomiędzy węzłami na stercie. Schemat połączeń dla przykładowej listy trzyelementowej pokazano na rysunku **??**.

3.4. Sposób używania narzędzi Git, GitHub oraz Doxygen

Obsługa systemu Git i platformy GitHub

System Git służy do zapisywania historii zmian w kodzie. Praca z narzędziem polega na zatwierdzaniu kolejnych etapów programu za pomocą commitów oraz wysyłaniu ich na serwer GitHub. Podstawowy sposób używania programu w terminalu obejmuje następujące kroki:

- `git init` oraz `git remote add origin <adres>` – utworzenie repozytorium i połączenie go z kontem na GitHubie,
- `git add .` oraz `git commit -m "opis"` – dodanie zmodyfikowanych plików do poczekalni i zapisanie zmian w lokalnej historii,
- `git push` oraz `git pull` – wysłanie commitów na serwer GitHub oraz pobranie aktualnej wersji kodu z serwera (np. po zmianach w innej lokalizacji na dysku),
- `git checkout <id_commita>` oraz `git reset --hard` – cofanie się do wcześniejszych wersji programu, usuwanie błędnego commita lub przywracanie usuniętych plików, aby projekt ponownie się kompilował.



Rys. 3.1. Diagram klas w projekcie listy dwukierunkowej

Obsługa generatora dokumentacji Doxygen

Program Doxygen automatycznie tworzy dokumentację na podstawie specjalnych komentarzy umieszczonych w plikach nagłówkowych (rozpoczynających się od `/**` i zawierających znaczniki takie jak `@brief` czy `@param`). Sposób użycia narzędzia składa się z trzech kroków:

1. Wygenerowanie pliku konfiguracyjnego poleceniem `doxygen -g`, gdzie w opcjach ustawia się nazwę projektu oraz włącza generowanie formatu LaTeX (`GENERATE_LATEX = YES`).

2. Uruchomienie programu poleceniem `doxygen Doxyfile`, co powoduje przeskanowanie plików `.h` oraz `.cpp` i utworzenie folderu `latex`.

3. Skompilowanie wygenerowanych plików w folderze `latex` za pomocą polecenia `make`, w wyniku czego powstaje gotowy plik wynikowy `refman.pdf` z pełną dokumentacją klas i metod.

4. Implementacja

W tym rozdziale przedstawiono kod źródłowy najważniejszych elementów programu oraz przebieg realizacji scenariuszy w systemie Git.

4.1. Implementacja węzła i wzorca Factory

Klasa `Node` przechowuje wartość elementu oraz wskaźniki na sąsiadów, natomiast klasa `ListFactory` odpowiada za tworzenie wszystkich obiektów na stercie za pomocą operatora `new`. Kod obu klas przedstawiono na listingu 1.

```
// --- Plik Node.h ---
#ifndef NODE_H
#define NODE_H

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val);
};
#endif

// --- Plik ListFactory.h ---
#ifndef LISTFACTORY_H
#define LISTFACTORY_H

#include "DoublyLinkedList.h"
#include "ListIterator.h"
#include "Node.h"

class ListFactory {
public:
    static DoublyLinkedList* createList();
```

```
26      static Node* createNode(int val);
27      static ListIterator* createIterator(DoublyLinkedList* list);
28  };
29  #endif
```

Listing 1. Pliki nagłówkowe Node.h oraz ListFactory.h

4.2. Implementacja klasy listy dwukierunkowej

Główna logika operacji na wskaźnikach znajduje się w pliku `DoublyLinkedList.cpp`. Na listingu 2 przedstawiono implementację dodawania i usuwania elementów na początku, na końcu oraz pod wskazanym indeksem.

```
1  void DoublyLinkedList::addFront(int val) {
2      Node* newNode = ListFactory::createNode(val);
3      if (!head) {
4          head = tail = newNode;
5      } else {
6          newNode->next = head;
7          head->prev = newNode;
8          head = newNode;
9      }
10 }
11
12 void DoublyLinkedList::addAt(int index, int val) {
13     if (index <= 0) {
14         addFront(val);
15         return;
16     }
17     Node* curr = head;
18     for (int i = 0; i < index - 1 && curr; i++) {
19         curr = curr->next;
20     }
21     if (!curr || !curr->next) {
22         addBack(val);
23         return;
24     }
25     Node* newNode = ListFactory::createNode(val);
26     newNode->next = curr->next;
27     newNode->prev = curr;
28     curr->next->prev = newNode;
29     curr->next = newNode;
30 }
31
32 void DoublyLinkedList::removeAt(int index) {
```

```
33      if (!head) return;
34      if (index <= 0) {
35          removeFront();
36          return;
37      }
38      Node* curr = head;
39      for (int i = 0; i < index && curr; i++) {
40          curr = curr->next;
41      }
42      if (!curr) return;
43      if (!curr->next) {
44          removeBack();
45          return;
46      }
47      curr->prev->next = curr->next;
48      curr->next->prev = curr->prev;
49      delete curr;
50  }
```

Listing 2. Wybrane metody z pliku DoublyLinkedList.cpp

4.3. Implementacja wzorca Iteratora i testów w funkcji main

Klasa `ListIterator` pozwala na poruszanie się po elementach listy i wyświetlanie kolejnego oraz poprzedniego węzła. Jej implementację oraz testy wszystkich metod w pliku `main.cpp` przedstawiono na listingu 3.

```
1   // --- Plik ListIterator.cpp ---
2   #include "ListIterator.h"
3   #include <iostream>
4   using namespace std;
5
6   ListIterator::ListIterator(Node* startNode) : current(startNode) {}
7
8   void ListIterator::showCurrent() const {
9       if (current) cout << "Aktualny element: " << current->data <<
        endl;
10      else cout << "Iterator jest pusty." << endl;
11  }
12
13  void ListIterator::showNext() {
14      if (current && current->next) {
15          current = current->next;
16          cout << "Nastepny element: " << current->data << endl;
17      } else {
```

```
18          cout << "Brak nastepnego elementu (koniec listy)." << endl;
19      }
20  }
21
22  void ListIterator::showPrev() {
23      if (current && current->prev) {
24          current = current->prev;
25          cout << "Poprzedni element: " << current->data << endl;
26      } else {
27          cout << "Brak poprzedniego elementu (poczatek listy)." <<
        endl;
28      }
29  }
30
31  // --- Plik main.cpp ---
32  #include <iostream>
33  #include "ListFactory.h"
34  using namespace std;
35
36  int main() {
37      DoublyLinkedList* list = ListFactory::createList();
38
39      list->addBack(10);
40      list->addBack(20);
41      list->addFront(5);
42      list->addAt(2, 15);
43      list->displayForward();
44      list->displayBackward();
45
46      ListIterator* it = ListFactory::createIterator(list);
47      it->showCurrent();
48      it->showNext();
49      it->showPrev();
50
51      list->removeAt(2);
52      list->removeFront();
53      list->removeBack();
54      list->displayForward();
55
56      list->clear();
57      delete it;
58      delete list;
59      return 0;
```

```
60 }
```

Listing 3. Implementacja iteratora (ListIterator.cpp) oraz funkcja main.cpp

4.4. Przebieg testów z systemem Git

Podczas pisania powyższego kodu przetestowano wszystkie wymagane scenariusze w systemie Git:

1. **Co najmniej 5 commitów:** każda klasa oraz kolejne metody były zatwierdzane osobnymi commitami (łącznie wykonano ponad 7 commitów).
2. **Cofnięcie się o dwa commity:** użyto polecenia `git checkout HEAD~2`, aby sprawdzić wcześniejszy stan projektu, a następnie powrócono na główną gałąź poleceniem `git checkout main`.
3. **Usunięcie jednego commita:** dodano próbny commit, a następnie usunięto go z historii za pomocą `git reset --hard HEAD~1`.
4. **Praca w dwóch lokalizacjach:** sklonowano repozytorium z GitHuba do drugiego katalogu (`git clone`), dopisano tam metodę `removeAt` i wysłano zmiany na serwer (`git push`). Następnie w pierwszym katalogu pobrano zmiany (`git pull`), dopisano klasę iteratora i ponownie wysłano na serwer.
5. **Usunięcie pliku i naprawa projektu:** celowo usunięto plik `Node.cpp`, przez co kompilator `g++` zgłosił błąd braku pliku. Następnie przywrócono ostatnią działającą wersję z repozytorium komendą `git checkout -- Node.cpp`, dzięki czemu program znów zaczął się kompilować.

5. Wnioski

Realizacja projektu pozwoliła na praktyczne przećwiczenie programowania obiektowego w języku C++ oraz obsługi narzędzi wykorzystywanych w codziennej pracy programisty.

- **Działanie listy dwukierunkowej na stercie:** samodzielna implementacja wskaźników `next` oraz `prev` pozwoliła dokładnie zrozumieć, jak działają dynamiczne struktury danych. Bardzo ważne okazało się pilnowanie zwalniania pamięci operatorem `delete` przy każdym usuwaniu węzła oraz w destruktorze listy.

- **Wzorce projektowe i podział na pliki:** rozbicie programu na osobne pliki `.h` oraz `.cpp` wymusiło odpowiednią konfigurację kompilatora `g++`, ale sprawiło, że kod stał się znacznie bardziej przejrzysty. Zastosowanie wzorca *Factory* uporządkowało tworzenie obiektów na stercie, a wzorzec *Iterator* ułatwił przeglądanie kolejnych i poprzednich elementów listy.

- **Praca z systemem Git i serwisem GitHub:** przetestowane scenariusze pokazały, jak bardzo przydatny jest system kontroli wersji. Nawet w przypadku przypadkowego usunięcia ważnego pliku z kodem lub wprowadzenia błędu, Git pozwala w kilka sekund cofnąć się do poprzedniej działającej wersji programu oraz wygodnie synchronizować pracę między różnymi folderami.

- **Dokumentacja w Doxygen i LaTeX:** użycie programu Doxygen pokazało, że odpowiednio opisany kod źródłowy pozwala wygenerować gotową dokumentację techniczną niemal automatycznie, co oszczędza bardzo dużo czasu.

Spis rysunków

3.1. Diagram klas w projekcie listy dwukierunkowej 8

Spis tabel

2.1. Podstawowe operacje na liście dwukierunkowej 5

Spis listingów

1. Pliki nagłówkowe Node.h oraz ListFactory.h 9
2. Wybrane metody z pliku DoublyLinkedList.cpp 10
3. Implementacja iteratora (ListIterator.cpp) oraz funkcja main.cpp . . . 11